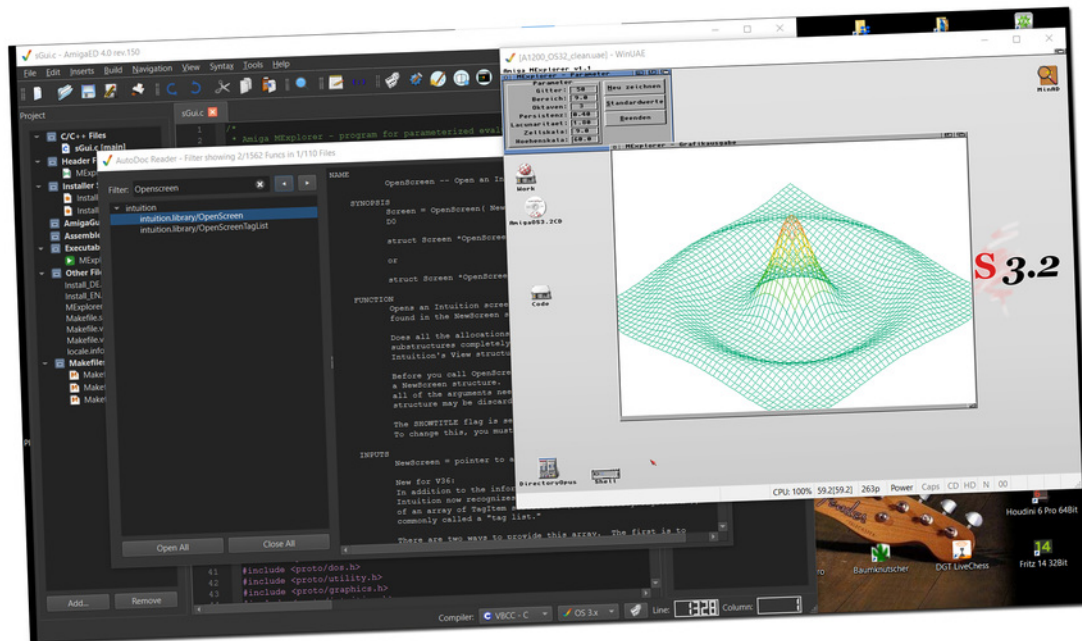


AmigaED 4.0

User Manual

Open Source Cross Compiler IDE for Bebbo's „amiga-gcc“
and its forks - works for VBCC, GCC and vasm/GNU as Assemblers



developed and maintained by Michael Bergmann

Table of Contents

1. Introduction
2. Two Ways to Build Your Code
3. Toolbar Reference
4. Mode 1: Ad-hoc One File Compilation
5. Mode 2: Project Driven Compilation
6. Quick Start: From Template to Clean Build
7. The Preferences Editor
8. The AutoDoc Reader
9. The Editor's Context Menu
10. The Compiler Output Pane

Introduction



AmigaED

Two Ways to Compile

A short, illustrated guide to Ad-hoc One File Compilation, Project Driven Compilation, and the Preferences Editor.

AmigaED is a cross-platform C/C++ and m68k Assembler IDE for classic Amiga development.

Two Ways to Build Your Code

AmigaED supports two independent ways to turn your source code into an AmigaOS-runnable program. Both are always available side by side, so use whichever fits what you are doing right now.

1. Ad-hoc One File Compilation – open a single source file and compile it directly, with no project setup at all. Best for a quick test, a one-off tool, or trying out an idea.

2. Project Driven Compilation – group multiple files (sources, headers, assembler, icons, and so on) into a Project. AmigaED tracks them, generates Makefiles for you, and builds the whole thing with one click. Best for anything with more than one file, a GUI (ReAction/MUI), or a program you will come back to later.

	Ad-hoc	Project
Best for	Quick single-file tests	Multi-file / GUI programs
Setup needed	None, just open a file	Create or import a project
Multiple files linked together	No	Yes
Makefiles	Not generated	Auto-generated (gcc, vbcc, SAS/C)
Persists between sessions	No	Yes (.aep project file)

Toolbar Reference

Every toolbar button mirrors a menu entry exactly - the toolbar is purely a shortcut for the entries you reach for most often, never a separate feature of its own.



#	Button	Menu equivalent	What it does
1	New	File → New	Opens a fresh, empty editor tab.
2	Open	File → Open...	Opens an existing file in a new tab.
3	Save	File → Save	Saves the current tab.
4	Save As	File → Save As...	Saves the current tab under a new name/location.
5	Print	File → Print file...	Prints the current file.
6	Undo	Edit → Undo	Undoes the last edit.
7	Redo	Edit → Redo	Redoes the last undone edit.
8	Cut	Edit → Cut	Cuts the current selection.
9	Copy	Edit → Copy	Copies the current selection.
10	Paste	Edit → Paste	Pastes the clipboard contents.
11	Search	(context menu) Search and Replace...	Opens the Find/Replace panel for the current tab.
12	Goto Line	Navigation → Goto Line...	Jumps straight to a given line number.
13	Matching bracket	Navigation → Goto matching bracket	Jumps to the bracket/brace/parenthesis matching the one under the caret - see “Bracket Matching” below for the red highlighting this also relies on.
14	Compile	Build → Compile...	Ad-hoc single-file compile (Mode 1) - see below.
15	Build Project	Build → Build Project	Project build (Mode 2) - see below.
16	Clean Project	Build → Clean Project	Removes the current project's build artifacts (object files, executable, icon).
17	AutoDoc Reader	Build → AutoDoc Reader...	Opens (or raises) the non-modal NDK AutoDocs browser - see “The AutoDoc Reader” below.
18	Open Shell	Build → Open Shell	Opens the system's default command line, starting in the current project's folder (or the configured “Projects root”

			if none is loaded).
19	Start Emulator	Tools → Emulator → Start default Workbench in UAE...	Launches your configured UAE emulator.
20	Stop Emulator	Tools → Stop running Emulation...	Stops the running emulator instance.
21	Exit	File → Exit	Closes AmigaED.

Warning: Open Shell gives you an unrestricted command line in the project's own folder - including the Makefiles and every file a build reads or produces. Editing, renaming, or deleting files there by hand, outside of AmigaED's own actions, can leave the project in a state that no longer matches what AmigaED expects, and may cause the next Build Project or Clean Project to fail or behave unexpectedly. Treat the shell as a power-user escape hatch, not part of the normal edit/build cycle.

Note: Start/Stop Emulator automatically grey out and enable each other depending on whether an emulator instance is currently running - including one AmigaED itself didn't start (e.g. left open on purpose from a previous session, or launched outside AmigaED entirely).

Choosing a Compiler or Assembler

Both the compiler dropdown in the status bar (bottom right) and Build → Select Compiler... in the menu bar drive the exact same choice - picking one always updates the other, since both funnel through the same central function. Five entries are available:

Status bar	Build → Select Compiler...	Used for
VBCC - C	VBCC vc (C mode only)...	Compiling C sources with vbcc.
GNU - C	GNU gcc (C mode)...	Compiling C sources with m68k-amigaos-gcc.
GNU - C++	GNU g++ (C++ mode)...	Compiling C++ sources with m68k-amigaos-g++.
vasm	vasm (Assembler mode)...	Assembling/linking a vasm-dialect Assembler Project (via vasm and vlink).
GNU as	GNU as (Assembler mode)...	Assembling/linking a GNU-as-dialect Assembler Project (via m68k-amigaos-as and m68k-amigaos-ld).

For an ad-hoc single file (Mode 1 above) or a C/C++ Project, only the three C/C++ entries make sense - picking vasm or GNU as there is refused with the warning “You can't compile your ASM-Project with a C-Compiler! Please choose vasm or GNU as.” and falls back to vasm. For an open Assembler Project (see “Creating an Assembler Project” in the Quick Start chapter above), it is the other way round: the project is locked to whichever of vasm/GNU as it was created for, and picking a C compiler or the other assembler dialect is refused the same way, falling back to the project's own locked-in choice.

The GNU as and GNU ld executable paths (needed for the GNU as entry above) are configured in Prefs → GCC, alongside the existing GCC/G++ fields - see “The Preferences Editor” below.

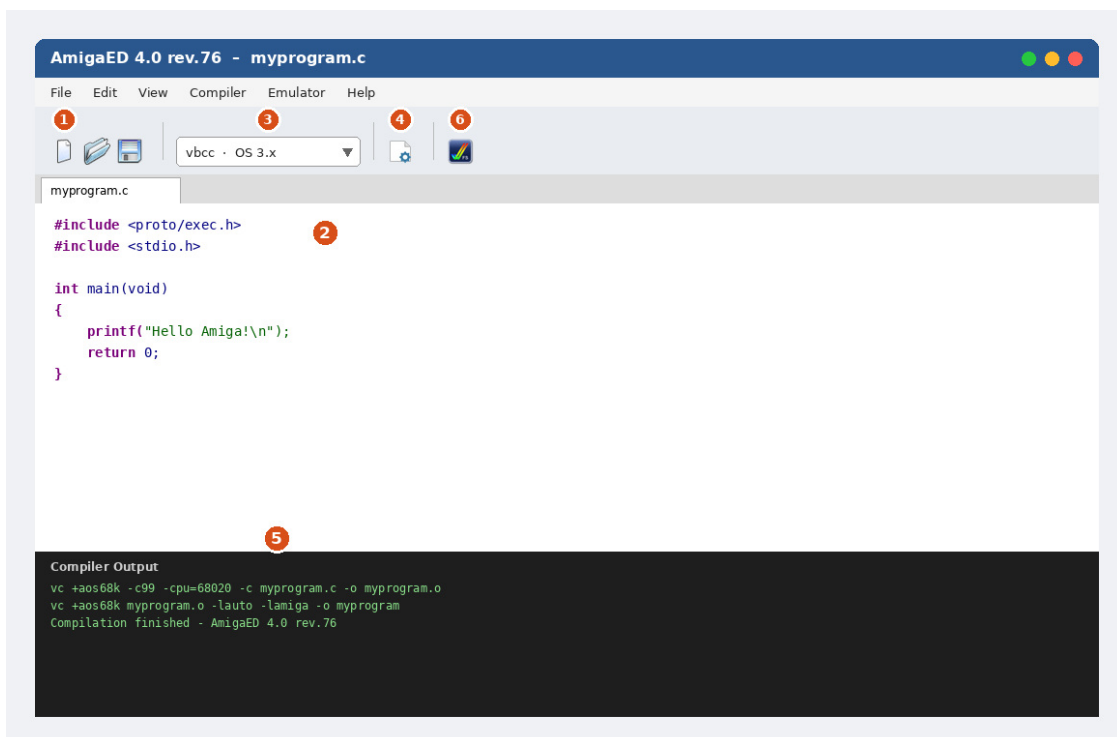
Bracket Matching

Whenever the caret sits directly before or after a bracket, brace, or parenthesis, both it and its matching partner turn red - this happens automatically as you type or move the caret, independently of the Navigation → Goto matching bracket command above, and regardless of whichever syntax colouring is currently active.

Tip: Prefs → Misc → “Highlight block between braces” (unchecked by default) additionally shades the text between a bracket pair with a subtle, readability-preserving background the moment you use Navigation → Goto matching bracket - handy for confirming at a glance that a long bracketed passage is self-consistent. This shading only appears for that command, never for the always-on red colouring described above, and it disappears immediately once the caret moves outside the bracket pair.

Mode 1: Ad-hoc One File Compilation

Use this when you just want to write and test a single source file, with no project, no linking multiple files, and nothing to set up first.



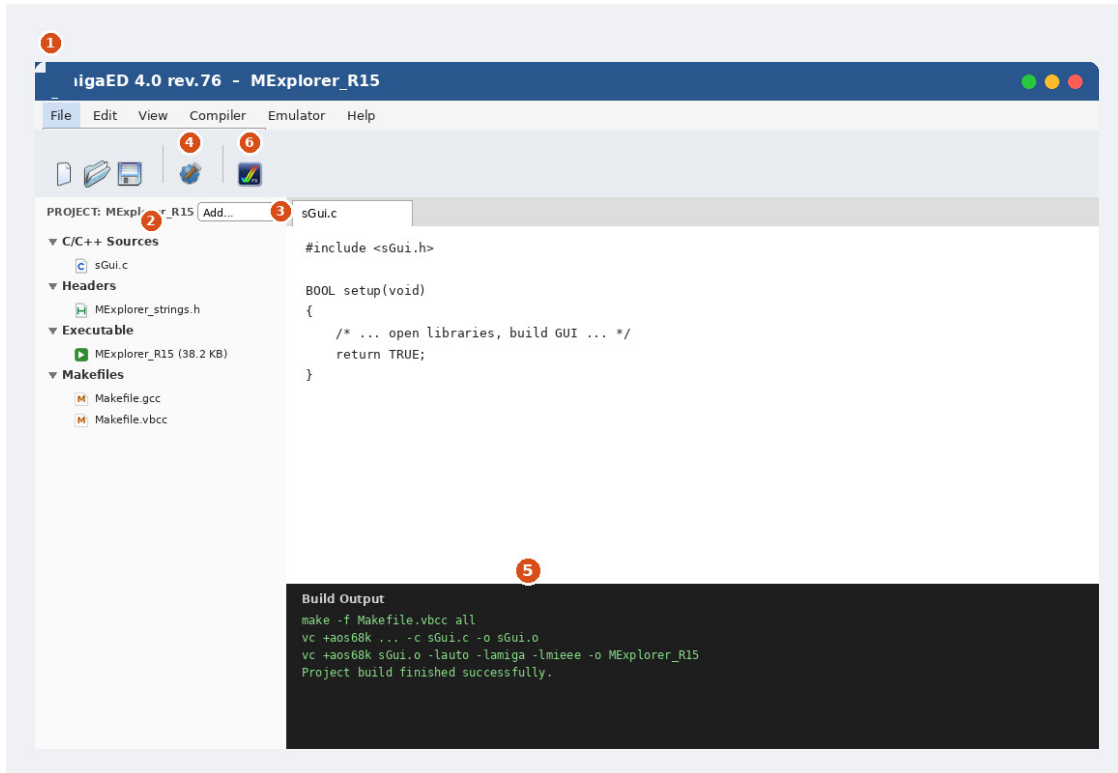
The editor in Ad-hoc mode: numbers match the steps below.

1. File → New (or Open...) to create or open a single .c/.cpp file. No project is involved.
2. Write or edit your code in the editor tab, with full syntax highlighting.
3. Pick your target toolchain and OS from the compiler dropdown in the toolbar (e.g. vbcc / OS 3.x, or m68k-amigaos-gcc / OS 1.3).
4. Click the Compile toolbar icon (or Compiler menu) to build just this file.
5. Check the Compiler Output panel at the bottom, which shows the exact compiler/linker commands used and whether the build succeeded.
6. Optionally, click Start Emulator to launch UAE and try your freshly built program right away.

Note: Ad-hoc compilation always builds and links just the one open file. If your program needs more than one source file, switch to Project Driven Compilation instead (see next chapter).

Mode 2: Project Driven Compilation

Use this for anything with more than one source file, a GUI (ReAction or MUI), or a program you will want to rebuild and maintain over time.



The Project panel and toolbar: numbers match the steps below.

1. File → New Project... to start from a template (Empty C, AmigaShell, AmigaOS 1.3, AmigaOS 3.x, ReAction, or MUI), or File → New Project... → Import existing Project... (Ctrl+Shift+I, also in the toolbar) to bring in a folder of sources AmigaED does not know yet.
2. The Project panel on the left shows all your files, automatically sorted into categories: C/C++ Sources, Headers, Assembler Sources, AmigaGuide, Installer Scripts, Executables, Other Files, and the auto-generated Makefiles.
3. Add further files any time with Add files to Project..., or by dragging them straight onto the Project panel.
4. Click Build Project. AmigaED (re)generates the Makefiles for every configured toolchain and runs the build for you.
5. Watch the Build Output panel. On success, the compiled program appears in the Project panel Executable category, together with its file size.
6. Click Start Emulator to launch UAE and test your program.

Note: A project's Makefiles are regenerated automatically every time you add or remove a file, and also right before every Build Project - so a plain content change (e.g. adding floating-point usage to an already-tracked file)

is always picked up too, not just file-list changes. Both vbcc and gcc/g++ Makefiles are generated side by side, plus a SAS/C Makefile (Makefile.sc) for smake - on a real Amiga, or under vamos with the compiler “SAS/C (vamos)”.

Note: Build Project also saves any open project file with unsaved changes first - including a hand-edited Makefile, not just source files - since both the Makefile regeneration above and the compiler itself always read straight from disk. Whether this happens automatically or asks first is controlled by Prefs → Project → “Save Project Files Automatically” (see below).

Closing a project

File → Close Project closes every tab belonging to the current project (prompting to save any with unsaved changes first, exactly like switching to a different project), then returns to the “no project loaded” state. Cancelling a save prompt leaves the project and all its tabs open untouched.

Automatic program icons

If Prefs → Misc → “create icon” is checked, AmigaED writes its own built-in, multi-coloured Amiga tool icon next to your compiled program - both for a project build and for an ad-hoc single-file compile. No default icon file needs to be configured anywhere. The icon's stack size is set automatically for the target: 4096 bytes for AmigaOS 1.3, 8192 for AmigaOS 3.x and Shell projects, 16000 for a ReAction project (template or GUI Builder), and 36000 for MUI, Empty C and imported projects.

Cleaning a project

Build → Clean Project removes the current project's build artifacts - every object file, the compiled executable, and its icon - by deleting each one directly and individually, listing every file it removes in the Compiler Output panel. It does not invoke “make clean” through a shell.

Running a program in vamos

vamos (from amitools) runs AmigaOS Shell programs directly on your PC - no emulator needed. Right-click the built program under “Executable” in the project tree and choose **Run in vamos**, or **Run in vamos with arguments...** to pass a command line (quoted arguments with spaces work as on the Amiga). The program's output and its return code appear in the Compiler Output pane. While it runs, the same menu offers **Stop vamos program**.

The program's own folder is mounted as the volume `amigaed:` and becomes the current directory, so it finds files next to itself. Set vamos up under Prefs → vamos: the vamos command (Linux: `vamos`; Windows: vamos runs inside WSL, e.g. `wsl ~/amitools-venv/bin/vamos` with “vamos runs inside WSL” checked, which translates `D:\x` into `/mnt/d/x`), optionally the host folders of your emulator's Workbench and Work partitions (mounted as `workbench:` and `work:`), and extra options (default `-m 8000 -s 256`).

Note: vamos has no windows or graphics, and its IEEE maths emulation is unreliable - test GUI and floating-point programs in the emulator. Programs get no console input (reading stdin returns end-of-file). Double-clicking a program of a Shell project runs it in vamos (it no longer opens the binary in the editor); GUI projects start in the emulator instead.

Building with SAS/C (vamos)

Build → **Build Project with SAS/C (vamos)** compiles and links the project's plain .c files with SAS/C 6.58 running under vamos - the same options as the generated Makefile.sc, plus BATCH so the linker never waits for keyboard input. It runs in the project folder, so the object files, the .lnk file and the program end up there. This works independently of the compiler selected in the status bar.

Needs, under Prefs → vamos: the Workbench folder (SAS/C is expected in workbench:SAS-C - change "SAS/C directory" if it lives elsewhere) and, for MUI projects, the Work folder (MUI is assigned from work:MUI). Errors and warnings are coloured and clickable like those of the cross compilers.

Note: If symbols are undefined, slink still writes a program in which they are replaced by a stub - running it would crash. AmigaED reports the build as failed and removes that incomplete program again.

Multilingual programs (FlexCat)

File → New Project → **New multilingual Amiga C Project** creates a Shell program whose texts come from a catalog via locale.library. Next to the main file it creates:

<Name>.cd - the catalog description with every text and its ID; the built-in language is English.

<Name>_deutsch.ct - the German translation. <Name>_strings.h - generated by FlexCat from the .cd (IDs, English texts and a lookup table); never edit it by hand. The main file opens locale.library and the catalog, and fetches every text through a small GetString() function - without locale.library or a matching catalog the program simply speaks English.

Makefile.gcc and Makefile.vbcc regenerate the header whenever the .cd changes and build Catalogs/deutsch/<Name>.catalog from the .ct on every build. For another language, add <Name>_<language>.ct (e.g. <Name>_francais.ct) to the project - AmigaED adds its catalog rule and folder automatically. On the Amiga, copy the Catalogs drawer next to the program (or the catalog to LOCALE:Catalogs/<language>/).

FlexCat is set under Prefs → Tools → "Multilingual Programs"; if that's empty, AmigaED uses a FlexCat next to m68k-amigaos-gcc. FlexCat's sd folder must sit next to the executable.

Note: .cd and .ct files are ISO-8859-1 (## codeset 0), like every file AmigaED saves. FlexCat copies the characters into the catalog unchanged, so umlauts appear correctly on the Amiga. FlexCat messages in the output pane are coloured and clickable (they jump to the line in the .cd/.ct file); FlexCat stops the build already on warnings such as "mismatching placeholders", because a wrong format string can crash the program. Makefile.sc has no FlexCat step - SAS/C uses the header generated by AmigaED.

Starting a program in the emulator

Right-click a built program under "Executable" and choose **Start in emulator (Workbench)** - it starts as if its icon had been double-clicked on the Workbench (real WBStartup message, ToolTypes and stack from the icon). A double-click in the project tree does the same for GUI projects (AmigaOS 3.x, ReAction, MUI, GUI Builder); Shell projects run in vamos instead. Both ways are always in the context menu.

If the emulator isn't running, AmigaED asks first and then starts it with the OS 3.x configuration; the program starts as soon as the Workbench is up. If it is running already - also one you started yourself - the program starts right away, without a reboot.

Requirements: projects lie in the projects root (Prefs → Project), and the projects root lies inside a **folder hard drive** of your emulator configuration (Prefs → Emulator, OS 3.x) - if it doesn't yet, add it in WinUAE/FS-UAE as a folder hard drive (e.g. volume Projekte). AmigaED reads the drives from the configuration and works out the Amiga paths itself; "Projects root on the Amiga" on the Project tab is only needed to override that (its folder button converts a picked folder into the Amiga path).

AmigaED also finds the Amiga's `S:User-Startup` on the boot drive of that configuration. On the first start in the emulator it offers to add a short block there that starts AmigaED's job runner from `<projects root>/AmigaED-Jobs` at every boot (also available as Prefs → Emulator → **Set up start script in S:User-Startup...**). A backup is kept in `User-Startup.bak`. If the emulator was already running at that moment, reboot the Amiga once.

Note: The double-click start uses the Workbench's ARexx port, so RexxMast must run (AmigaOS 3.2 starts it by default). A program without an icon is started with `WBRUN` instead (`C:WBRUN` plus `parm.library`). The output pane shows when the Amiga has started the program, or what went wrong.

SAS/C (vamos) as the selected compiler

Build → Select Compiler and the compiler box in the status bar have a sixth entry, **SAS/C (vamos)**. It is greyed out until AmigaED finds both the `vamos` command and `SAS/C` (`c/sc` and `c/smake` in the "SAS/C directory", Prefs → `vamos`, default `workbench:SAS-C` inside the Workbench folder set there). AmigaED checks this at start-up and whenever the Prefs are closed; the status bar tip of the greyed-out entry says what is missing. Loading a project selects the default compiler from the Prefs again, as with the other compilers. `SAS/C (vamos)` can itself be that default: Prefs → Misc → "Default cross compiler" offers it as well (greyed out there too while it isn't available - checked live against the entries on the `vamos` tab). If the default can't be used at start-up, AmigaED says why and starts with `VBCC - C`.

With `SAS/C (vamos)` selected:

Dice button / F6 compiles and links the file in the current tab in its own folder - `sc <options> BATCH <name>.c LINK TO <name>_sc`, named like the single-file results of the other compilers (`_vc`, `_gcc`). A file of the current project gets the project's `SAS/C` options (the same as in `Makefile.sc`); any other file gets the options from Prefs → `SAS/C`, plus `MATH=IEEE` if it uses `float/double` and the MUI include folder if it includes a MUI header. Only plain `.c` files can be compiled.

Build Project / Shift+F6 runs the project's `Makefile.sc` with `SAS/C`'s own make tool, exactly as on the Amiga: `smake -f Makefile.sc` in the project folder. `smake` only rebuilds the program if a source file is newer. A hand-edited `Makefile.sc` is used as it is; the program name for the result check comes from its `TARGET` = line. Clean Project also removes the `.lnk` file `SAS/C` writes.

Note: Errors and warnings are coloured and clickable as usual. If symbols are undefined, the build is reported as failed and the incomplete program removed. `smake` opens `icon.library`, which `vamos` does not have - AmigaED gives it a dummy library (`-O icon.library=mode:fake`). The separate entry Build → **Build Project with SAS/C (vamos)** remains: it calls `sc` directly with all the project's `.c` files, without `Makefile.sc` and whatever compiler is selected.

Fonts

Prefs → Misc → **Fonts** sets two fonts independently: **GUI font** for menus, dialogs, the project tree and the compiler output, and **Editor font** for the source editor (every tab and every syntax style). **Choose...**

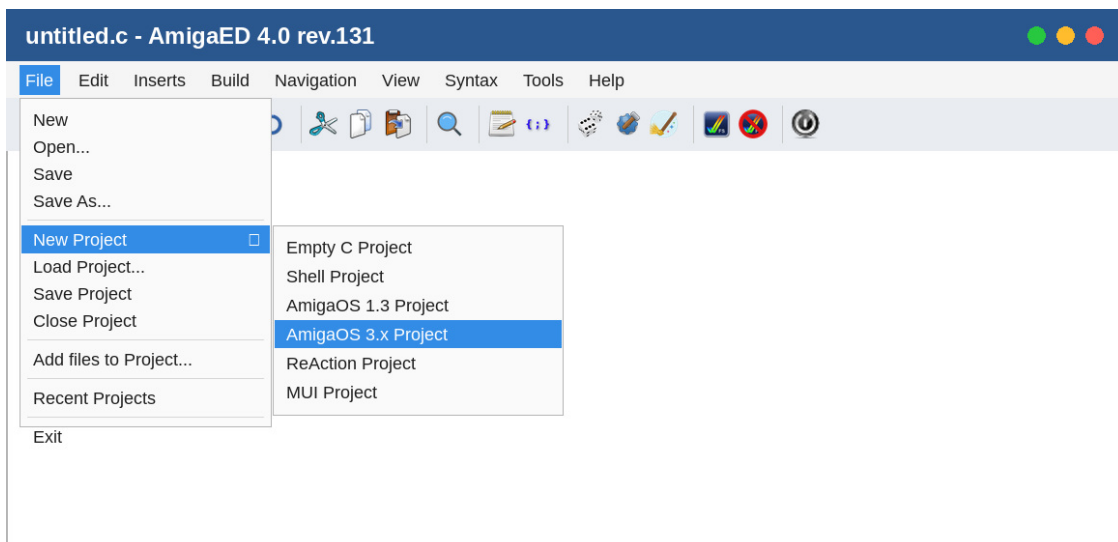
opens the font requester; with **Fixed font** checked (the default) the editor's requester only lists fonts with a fixed character width - uncheck it to pick any font. Each field shows the chosen font in its own typeface; **Default** goes back to the system font (GUI) or AmigaED's built-in monospaced font (editor). Both take effect as soon as the Prefs are saved - no restart needed; the editor's zoom level is kept.

Quick Start: From Template to Clean Build

A concrete, illustrated walk-through of Mode 2 above, start to finish: creating a new project from a template, building it, and cleaning it up again.

1. Create the project

File → New Project offers six templates: Empty C Project, Shell Project, AmigaOS 1.3 Project, AmigaOS 3.x Project, ReAction Project, and MUI Project. The ReAction and MUI templates already build a small working window with a File menu (About/Quit) - a genuine starting point, not just an empty shell.

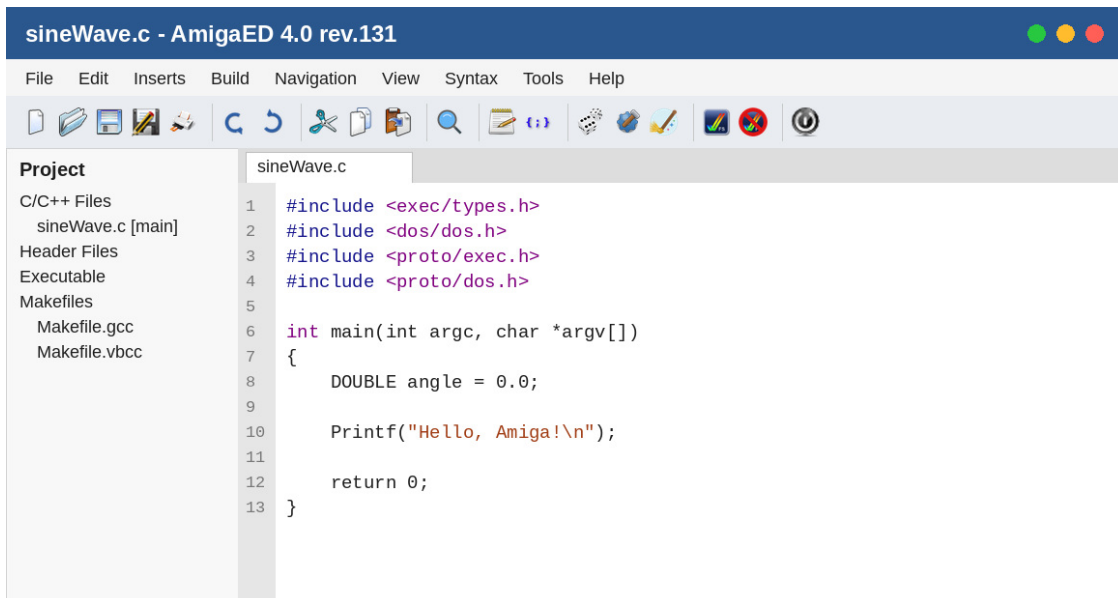


Picking "AmigaOS 3.x Project" from the New Project submenu.

After picking a template, AmigaED asks for a target directory and a project name, then (for AmigaOS 1.3/3.x, ReAction, and MUI templates) confirms the default compiler/linker switches for that target before creating anything - the same switches listed in the table further below, pre-filled and ready to adjust.

2. Write your code

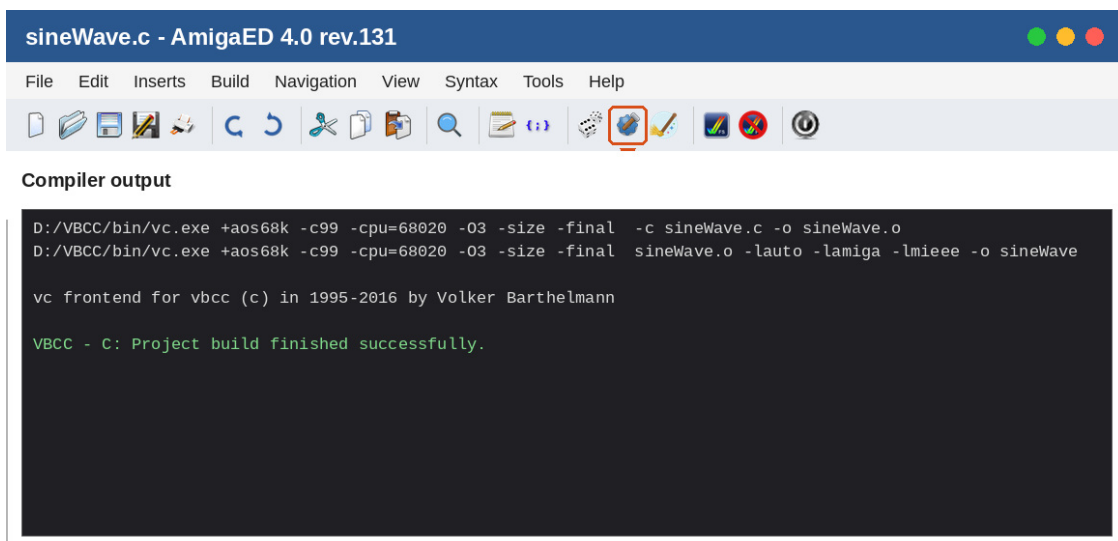
The generated main file opens automatically, already including the right Amiga NDK headers for the chosen template. The Project panel on the left shows the file, the auto-generated Makefiles, and the (still empty) Executable category.



A freshly created AmigaOS 3.x project, ready to edit.

3. Build it

Click the Build Project toolbar icon (or Build → Build Project). AmigaED regenerates the Makefiles fresh, then runs the currently selected compiler (VBCC or GCC/G++, picked in the status bar) against them. If your code uses `float/double` - or the Amiga-typical uppercase `FLOAT/DOUBLE` typedefs, both are recognized - the right math library is linked in automatically; nothing to configure by hand.

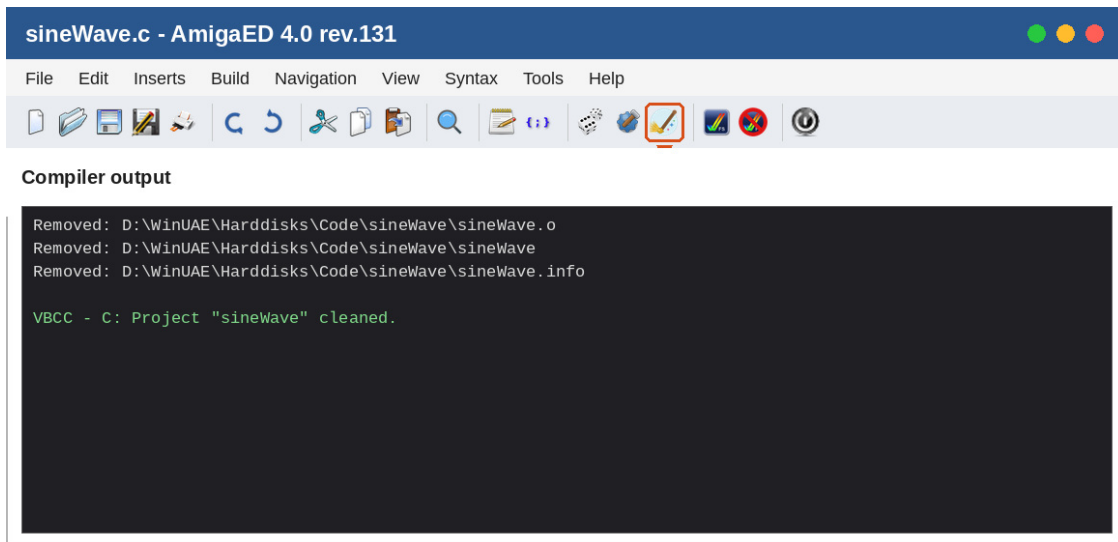


A successful build, reported in the Compiler Output panel.

See “The Compiler Output Pane” below for how to jump straight from an error or warning line to the offending source line.

4. Clean up

Click the Clean Project toolbar icon (or Build → Clean Project) to remove the build artifacts again - every object file, the compiled executable, and its icon - each one deleted directly and listed individually in the Compiler Output panel.

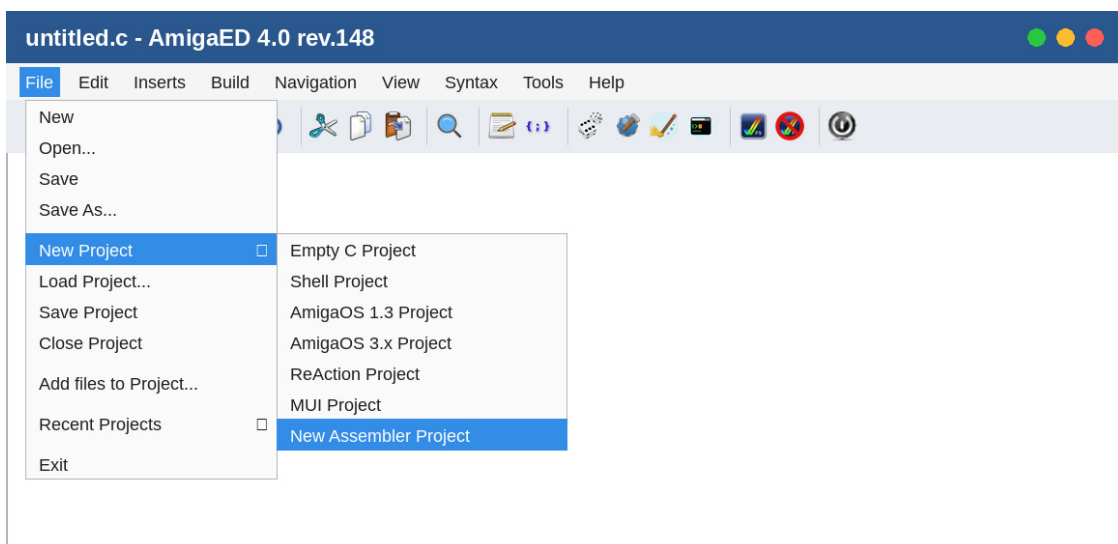


Clean Project's output: every removed file is listed by name.

Note: Your source files, the .aep project file, and the Makefiles themselves are never touched by Clean Project - only the files a build actually produces.

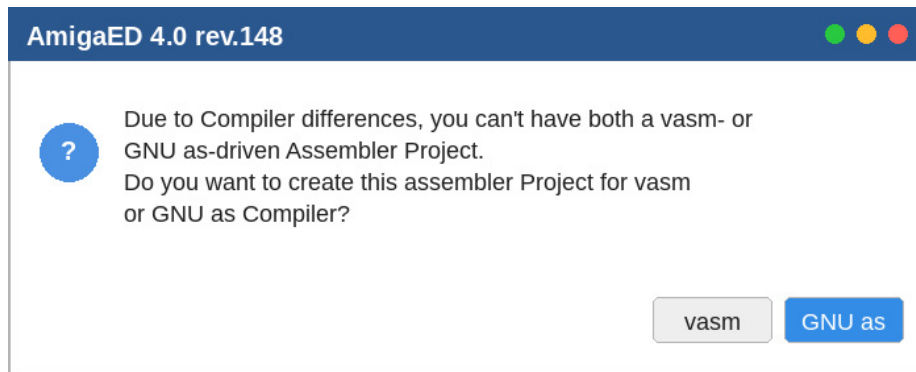
5. Creating an Assembler Project

New Assembler Project is a pure m68k assembler template - no C/C++ at all - and works a little differently from the other six templates above, because AmigaED supports two mutually incompatible assembler dialects: vasm and GNU as (m68k-amigaos-as). Rather than guessing or mixing them, AmigaED asks up front which one this project is for, and locks that choice in for the project's whole lifetime.



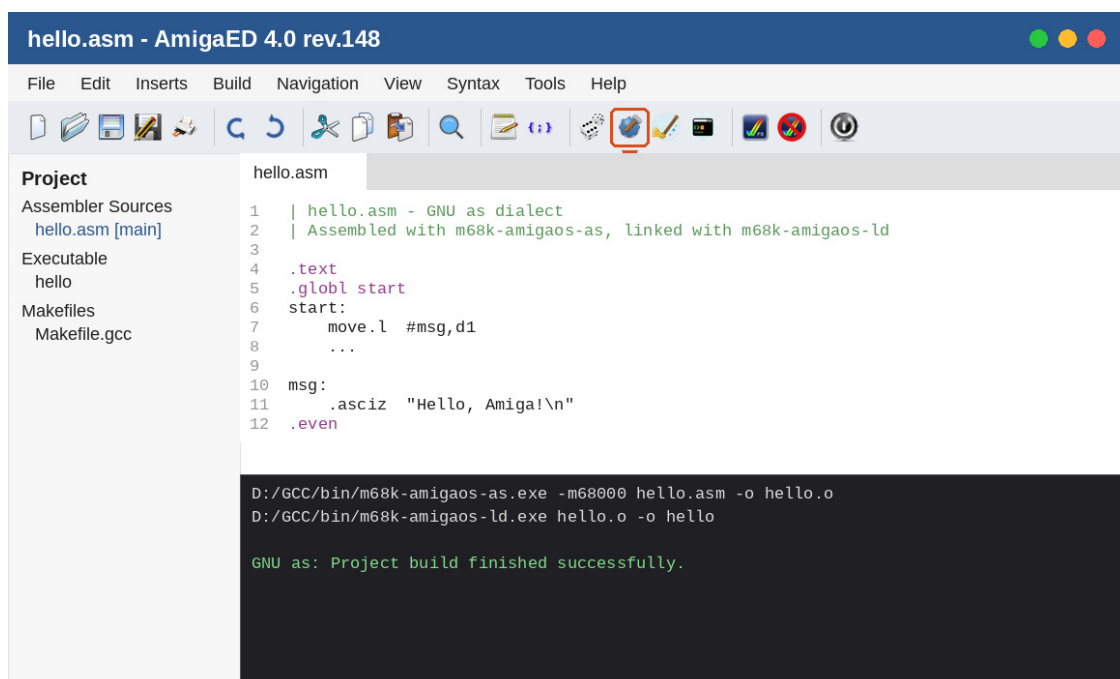
Picking "New Assembler Project" - the seventh, newest template.

The very first thing AmigaED asks - before the usual directory and project name prompts - is which assembler this project is for:



"Due to Compiler differences, you can't have both a vasm- or GNU as-driven Assembler Project. Do you want to create this assembler Project for vasm or GNU as Compiler?"

Whichever button you click decides everything that follows: AmigaED generates only the matching Makefile (Makefile.vbcc for vasm, Makefile.gcc for GNU as - never both), writes the freshly generated main .asm file in that dialect's own comment and directive style, and switches the statusbar compiler chooser (and Build → Select Compiler...) to match automatically.



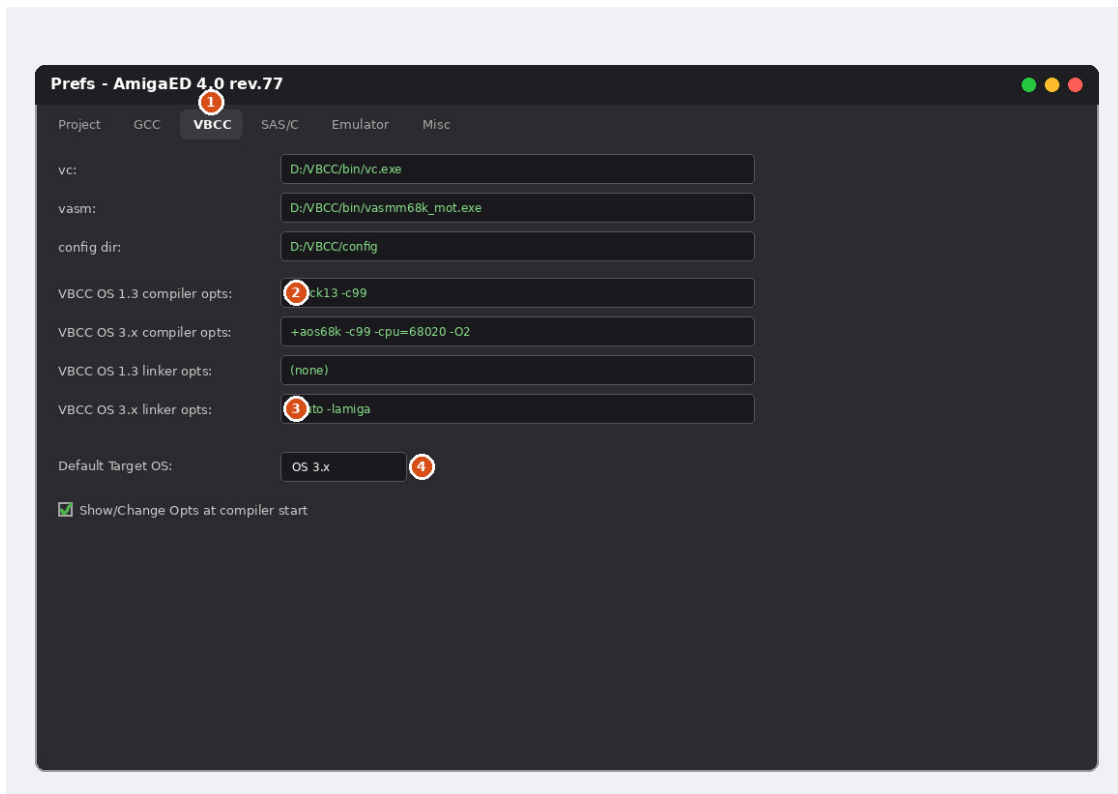
A GNU as Assembler Project after Build Project: assembled with m68k-amigaos-as, linked with m68k-amigaos-ld (its path set in Prefs → GCC → "GNU ld:") - only Makefile.gcc was ever generated for it.

Note: Both dialects are fully supported thereafter, each with its own correctly highlighted comment style in the editor: vasm accepts ";" and "*" as comment characters anywhere on the line, while GNU as uses "|" anywhere and "*" / "#" only at the very start of a line (in GNU as, ";" is a statement separator, not a comment). Picking a C compiler (VBCC/GNU C/GNU C++) or the assembler dialect this project was NOT created for - in either the statusbar or Build → Select Compiler... - is refused with a warning message box and automatically falls back to the project's own locked-in choice, so a mismatched build can never be triggered by accident.

Note: Reopening an existing Assembler Project - via File → Load Project... or File → Recent Projects - automatically switches the compiler chooser to that project's own locked assembler, even if a different project (or a different dialect) was active just before.

The Preferences Editor

File → Prefs... opens AmigaED settings, organized into tabs: Project (author/email/website, default projects folder), GCC, VBCC, and SAS/C (paths to each toolchain and their compiler/linker options, with separate fields for AmigaOS 1.3 and AmigaOS 3.x targets), Emulator (path to your UAE executable, and separate OS 1.3/3.x configs), and Misc (application style/theme, whether to create a program icon, and other editor behaviour).



The VBCC tab, as an example: GCC and SAS/C follow the same pattern.

1. Tabs across the top switch between Project, GCC, VBCC, SAS/C, Emulator, and Misc settings.
2. Compiler options are kept separate for AmigaOS 1.3 and AmigaOS 3.x, since the two targets typically need different flags (see the tables below).
3. Linker options are likewise separate per target OS. This is where a math library gets added automatically when your project uses floating point (see note below).
4. Default Target OS sets which of the two option sets above is used when you do not explicitly pick one.
5. The GCC tab additionally has “GNU as:” and “GNU ld:” fields (each with its own path field and file selector, right next to the existing gcc/g++ ones) - the assembler and linker used for a GNU-as-dialect Assembler Project, assembling and linking it directly rather than through gcc.
6. The GCC and VBCC tabs each also have an “Assembler Include Path:” field - when set, it is appended as `-I<path>` to that toolchain's assemble rule in the generated Makefile (vasm on Makefile.vbcc, GNU as on Makefile.gcc), so a hand-written .asm/.s source can `include/.include` files from there - e.g. the NDK's own separate assembler-include tree - without a hardcoded absolute path in the source itself. Left empty, no extra flag is added at all.

Automatic floating-point handling: if AmigaED detects float or double - or the Amiga-typical uppercase FLOAT/DOUBLE typedefs, both are recognized - anywhere in your project's own C/C++ sources, it automatically adds the right math library to the generated Makefiles for you: -lm for gcc/g++, -lmieee for vbcc, and MATH=IEEE for SAS/C (only if no MATH= mode is already set). You do not need to add these yourself.

Saving before a build

Prefs → Project → “Save Project Files Automatically” (directly below “Projects root”) controls what Build Project does when an open project file - a source file or a hand-edited Makefile alike - has unsaved changes: checked, it saves everything immediately with no prompt; unchecked (the default), it asks once, listing every affected file, with the option to save them, build anyway from whatever is already on disk, or cancel the build entirely.

Detecting Externally Changed Files

Whenever AmigaED regains focus - you switch back to it from another application - it checks every currently open file against what's actually on disk. If one or more have changed outside AmigaED (another editor, a build tool, version control, ...), a dialog lists every affected file as a pre-checked box, so you can reload some or all of them from disk in one go. A file that also has unsaved changes of your own is listed too, but pre-unchecked and flagged with a warning, since reloading it would silently discard those changes - you have to tick it deliberately.

Themes

Prefs → Misc → “Default application style” (or the View → Theme menu, which changes it immediately without opening Prefs at all) lists every native style available on your platform, plus three synthetic ones: **Dark**, **Workbench 1.3** (the classic blue Kickstart 1.3 desktop look, sampled from a real screenshot), and **Workbench 3.1** (the plainer, neutral grey AmigaOS 3.x look, also sampled from a real screenshot). View → Theme shows all of them as a single, mutually exclusive list - picking one applies it immediately, everywhere: the application chrome, every open editor tab, and the Compiler Output pane's error/warning colours all update together.

The emulator and running instances

If you close AmigaED while its emulator is still running, it asks whether to leave the emulator open or shut it down too - handy since starting an emulator takes real time you can skip on the next session. Starting the emulator also checks, via the operating system's own process list, whether a matching emulator process is already running - whether AmigaED itself started it (e.g. one left open like this) or not - and asks before launching a second instance rather than doing so unconditionally. Start/Stop Emulator (toolbar and Tools menu) automatically reflect whichever of these is currently true.

Known issue: FS-UAE under WSL2/WSLg: if you run the Linux build of FS-UAE inside WSL2 (Windows Subsystem for Linux), its mouse capture is unreliable - both the host and the emulated Amiga cursor can remain visible at once, drifting apart as you move the mouse, no matter how FS-UAE's own configuration options for mouse grabbing are set. This traces back to a known WSLg limitation with relative mouse input (see microsoft/wslg issue #240), not a bug in FS-UAE, AmigaED, or your configuration - confirmed by the same FS-UAE configuration working correctly on a real Linux machine. If you need FS-UAE specifically inside WSL2, running an X server on the Windows side (e.g. VcXsrv) instead of relying on WSLg's own GUI integration is the more

promising workaround; otherwise, running FS-UAE on a native Linux machine (or natively on Windows, using the Windows build of FS-UAE) avoids the issue entirely.

Editing UAE config files directly

The “OS 1.3 Config” and “OS 3.x Config” fields on the Emulator tab each have their own Edit button next to the usual file picker, opening that UAE config file directly in the platform's native text editor (Notepad on Windows, TextEdit on macOS, and the first available of gedit/kate/mousepad/leafpad, or a generic xdg-open fallback, on Linux) - handy for hand-tuning a setting UAE's own GUI does not expose.

NDK AutoDocs folder

The Emulator tab also has an “AutoDocs folder:” field (directory selector) pointing at your Amiga NDK's own “Autodocs” drawer - set this once to use Build → AutoDoc Reader..., described in its own chapter right after this one.

The AutoDoc Reader

Build → AutoDoc Reader... (also in the toolbar, right before Open Shell) opens a non-modal browser for the AmigaOS NDK AutoDocs - the plain-text *.doc files under the NDK's own “Autodocs” drawer (exec.doc, dos.doc, graphics.doc, and so on, one file per library), each holding every documented function of that library back to back. It lets you browse and full-text-filter every documented NDK function without ever leaving AmigaED, inspired by (though not a pixel clone of) the classic Workbench tool MinAD.

Prerequisite: Set Prefs → Emulator → “AutoDocs folder:” to your Amiga NDK's “Autodocs” drawer first (see “NDK AutoDocs folder” in the previous chapter). If that field is empty or no longer points at a real folder, AutoDoc Reader... shows a warning naming exactly where to set it, instead of opening an empty window.

1. Every *.doc file found (recursively) under the configured AutoDocs folder is parsed into individual function entries and grouped into a tree, one top-level group per source file - e.g. every function documented in exec.doc groups under “exec”.
2. Type into the Filter field to narrow the tree to matching function names as you type; Prev/Next step through matches, and Enter in the filter field jumps to the first/next match. The window title keeps a running “Filter showing X/Y Funcs in A/B Files” count while a filter is active.
3. Open All / Close All expand or collapse the whole tree in one click.
4. Selecting a function shows its complete, unmodified AutoDoc text on the right, in a guaranteed-monospace font - needed for the SYNOPSIS section's column-aligned register names (e.g. D0, A0) to line up correctly, exactly as in the original documentation.
5. Every SEE ALSO line in an entry's text is scanned for other documented function names, each turned into a clickable link - clicking one jumps straight to that function's own entry, exactly as if you had typed it into the Filter field yourself. When the same short name exists in more than one library (e.g. CMD_WRITE in audio.device, carddisk.device, and trackdisk.device), the link prefers the entry belonging to the SAME library as the one you clicked from, since that is what a SEE ALSO reference means by AmigaOS convention.

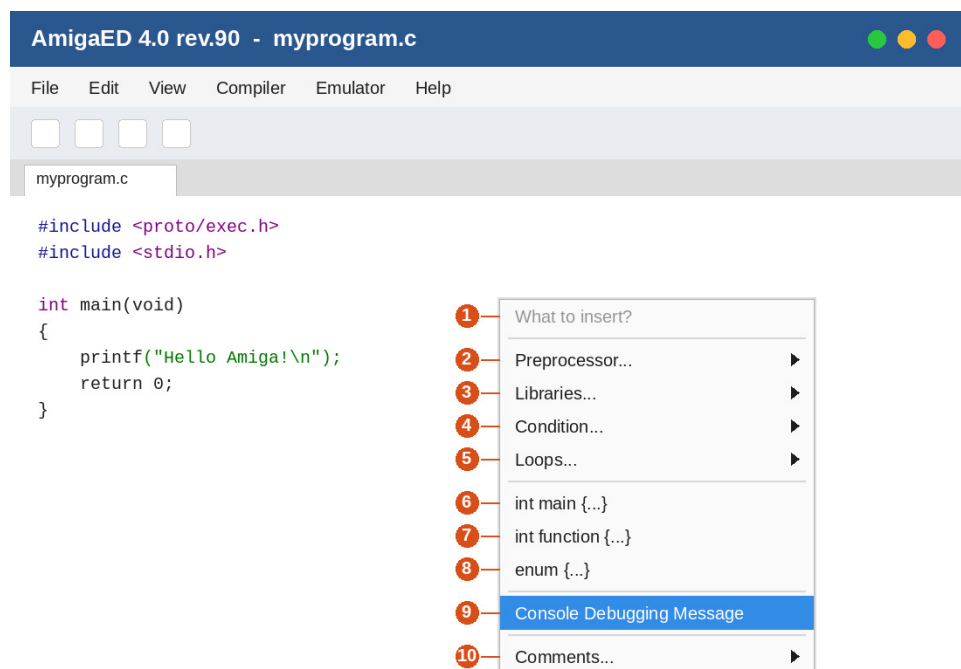
Note: AutoDoc Reader can only ever be open once, exactly like Help → Manual - triggering Build → AutoDoc Reader... again while it is already open simply raises the existing window instead of opening a second one (un-minimizing it too, if it was minimized). It is non-modal throughout, so the rest of AmigaED - editing, building, everything - stays fully usable while it is open, and it remembers its own window size and position the next time you open it. It can be resized from any edge or corner, same as any other window; the small grip in its bottom-right corner is just a visual reminder of that.

Tip: Instead of opening AutoDoc Reader and typing a function name into the filter by hand, right-click that name directly in the editor and choose **Jump to Explanation** - see “The Editor’s Context Menu” below. It opens (or raises) this same window and jumps straight to that function’s entry.

The Editor’s Context Menu

Right-click anywhere inside the code editor to open a context menu. Its lower section mirrors the main Inserts menu (menu bar) exactly: anything listed there is also available here, under the same name. Above that, three quick-access entries sit at the very top of the menu, not part of Inserts:

- **Comment/Uncomment Block** – toggles line-comment markers on the current selection (or the current line, with nothing selected), using whichever comment style the active syntax uses.
- **Search and Replace...** – opens the Find/Replace panel for the current tab, pre-filled with whatever word was under the click (see row 11 of the Toolbar Reference table above).
- **Jump to Explanation** – opens (or, if already open, raises and reuses) the AutoDoc Reader and jumps it straight to the entry for the NDK/MUI function under the click - e.g. right-clicking `OpenWindow` and choosing this jumps straight to `intuition.library/OpenWindow`, instead of having to open the AutoDoc Reader yourself and type the name into its filter by hand. Needs Prefs → Emulator → “AutoDocs folder:” set first (see “The AutoDoc Reader” above); if the word under the click isn’t a documented function, or nothing was under it at all, a status-bar message says so instead of opening an empty or unrelated entry.



The editor context menu's Inserts-mirrored section, opened over a source file: numbers match the entries explained below. The three quick-access entries above are shown further up in the real menu, ahead of this section.

1. **What to insert?** – a disabled heading line, not a clickable entry. It is only there to label the menu.
2. **Preprocessor...** – a submenu of preprocessor-related inserts: `#include`, `#define`, `#ifdef`, `#ifndef`, `#if defined(...)`, **Amiga #include files** (a ready-made block of the most common Amiga NDK headers), **Identify Amiga compiler** (a chain of `#if defined(...)` checks generating a `compiler_string` constant naming whichever compiler is in use), and **Amiga C version string** (a `$VER:` tag).
3. **Libraries...** – `OpenLibrary()` and `CloseLibrary()`, each inserting a ready-to-edit `dummy.library` Open/Close template with error handling.
4. **Condition...** – `if(..) {...}` and `if(..) {...} else {...}` skeletons.
5. **Loops...** – `while(...) {...}`, `for(...) {...}`, `do...{...}while(...)`, and `switch(...)` (already including a case `dummy: break;`).
6. **int main {...}** – inserts a complete `main()` function skeleton; disabled once the file already contains one.
7. **int function {...}** – inserts a function prototype plus a matching function skeleton, ready to cut and paste into place.
8. **enum {...}** – inserts a complete C enumeration skeleton (`SomeEnum` with three example values).
9. **Console Debugging Message** – inserts an `if (myDebug) {...}` debugging block; the caret lands right inside it, ready to type. Needs `#define myDebug TRUE` earlier in the file, already included near the top of every "New Project" template.
10. **Comments... – Fileheader comment...**, C-style single/multi line comments, a C++ style single line comment, and a C-style line-dividing comment.

Note: Most entries insert their skeleton right at the caret position and leave the caret ready to keep typing, either on a new line after the inserted block, or (for Console Debugging Message) right inside it. This context menu is built from the exact same actions as the main Inserts menu, so the two are always in sync.

The Compiler Output Pane

Every compile - ad-hoc (Mode 1) or project (Mode 2), VBCC or GCC/G++ - prints its output into the Compiler Output pane at the bottom, with two things making it easier to work through: colour-coded diagnostics, and click-to-jump.

Colour-coded errors and warnings

Every recognized error line is highlighted in red, every warning in yellow/orange - bold text on a tinted background, using a colour scheme matched to the current theme (see Themes above) so it stays readable whichever one is active. Purely informational lines (compiler invocation echoes, "In file included from...", GCC's own source-snippet/caret lines) are deliberately left uncoloured.

Click to jump

Click any error or warning line to jump straight to the offending file and line: the file opens in a tab (or is

simply activated, if already open) and the caret moves to the exact reported line and column.

Compiler	Recognized format
VBCC	error 9 in line 24 of "file.c": message
GCC/G++	file.c:24:5: error: message
vasm	error 9 in line 24 of "file.asm": message
GNU as	file.asm:24: Error: message

Note: An error reported inside a generated intermediate file (e.g. VBCC reporting an assembler-level problem in the .asm file it produced from your .c source) still jumps correctly - to that intermediate file, exactly as the compiler itself reported it.

Context menu

Right-click inside the pane for Qt's usual Copy/Select All entries, plus two AmigaED-specific ones: **Mark all and copy** selects the entire pane and copies it to the clipboard in one click, instead of the usual Select All + Copy two-step; **Empty Console** clears the pane outright - unlike a normal compile run, which only ever appends new output on top of whatever is already there.

Suggested Compiler & Linker Switches

These are AmigaED own built-in defaults: a solid, well-tested starting point for each toolchain and target. You are always free to add your own project-specific flags in the same Prefs fields alongside them.

m68k-amigaos-gcc (GCC/G++)

OS 1.3 compiler	-Wall -O2 -mcrt=nix13	Common warnings, optimize, link against libnix built for Kickstart 1.3.
OS 3.x compiler	-Wall -O2 -noixemul	Common warnings, optimize, use the AmigaOS-native C runtime instead of ixemul.library.
OS 1.3 linker	(none)	The OS 1.3 template is console-only and needs no extra libraries by default.
OS 3.x linker	-lamiga	Needed for Workbench-capable programs (ReAction/MUI), added automatically by AmigaED OS 3.x/ReAction/MUI templates.

vbcc (vc)

OS 1.3 compiler	+kick13 -c99	Named vbcc config for Kickstart 1.3, C99 language mode.
	+aos68k -c99	

OS 3.x compiler		Named vbcc config for AmigaOS 2.0+/3.x, C99 language mode.
OS 1.3 linker	(none)	Console-only OS 1.3 template needs no extra libraries by default.
OS 3.x linker	-lauto -lamiga	vbcc own amiga.lib equivalent, needed for Workbench-capable (ReAction/MUI) programs.

Optional additions many users like to add for a release build once things compile cleanly: -cpu=68020 (or higher), -O2/-O3 (optimize), -size (optimize for code size), -final (disable runtime checks). These are not part of AmigaED own defaults since they trade off compatibility or debuggability for speed/size.

SAS/C

SAS/C only runs on a real Amiga or emulator, so AmigaED never invokes it itself: it just generates a Makefile.sc alongside the others for later, manual use there. The SCOPTS field in Prefs starts empty; AmigaED still adds MATH=IEEE automatically if your project needs it.

Recommendations for Amiga C Programmers

A short, curated list of literature and tools the Amiga C/ReAction/MUI development community particularly recommends. None of this is bundled with AmigaED, but all of it pairs well with it.

Literature

Amiga ROM Kernel Reference Manual: Exec / Libraries and Devices / Includes and Autodocs

The original Commodore-Amiga reference series, still the authoritative source for Exec, Intuition, and the rest of the classic system libraries. Out of print; free scanned copies are archived at archive.org.

Amiga ROM Kernel Reference Manual: AmigaDOS – Thomas Richter

A modern, RKRM-style reference specifically for AmigaDOS/dos.library. Free download from Aminet; a printed edition has also been available via lookbehindyou.de.

ROM Kernel Reference Manual: Changes & Additions – Camilla Boemann & Jason Stead

An ongoing (WIP) volume covering everything added to the Amiga OS from Release 2 through 3.2. Free PDF from developer.amigaos3.net.

Erik Bartmanns Amiga Programmierbuch

C programming, cross-compiling with vbcc and ReAction, on an Amiga 4000 (or Amiga Forever). Book: 32 EUR, e-book: 19.99 EUR, via bombini-verlag.de.

Erik Bartmanns Amiga 1200 Buch

Machine language, C programming, and Shell: 717 pages across 28 chapters, focused on the Amiga 1200. Book: 64 EUR, e-book: 29.99 EUR, via bombini-verlag.de.

Tools

Rebuild – Darren Coles

A ReAction GUI builder (a from-scratch remake of the classic ClassMate), generating C or Amiga E code. Public domain. Source: github.com/dmcoles/ReBuild

Codecraft – Camilla Boemann

Codecraft is a powerful IDE for developing software natively on the Amiga. Codecraft makes it easy for you as a developer to write code, then build, execute and debug your resulting program. And everything is at your fingertips in one unified user interface. A native IDE for AmigaOS 3.2.3 and later. Source: gitlab.com/boemann/codecraft, <http://boemann.dk/codecraft/>

MUIBuilder

A user interface designer for the MUI and Zune toolkits. GPLv3/LGPLv3. Source: sourceforge.net/projects/muibuilder

Prices, links, and availability change over time. If something above is no longer reachable, a web search for the title and author is usually the fastest way to find its current home.